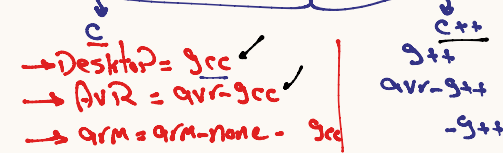
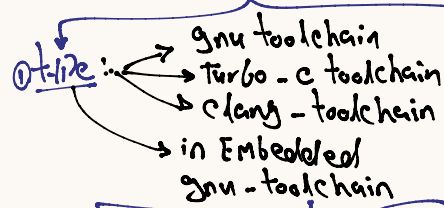


* session 9 : Pre Processor

* toolchain Parts



② Parts

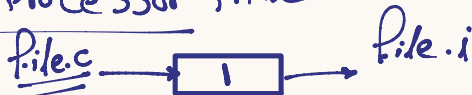


- ① Pre Processor
- ② Compiler
- ③ Assembler
- ④ linker

* Tool chain Part



① Pre Processor Phase



* text Replacement for Pre Processor directives

#include #if #elif
#define #else #endif
#undef #warning
#error #ifndef #ifdef

* gcc -E File.c -o File.i

② Compiler Phase



* Compiler doing these points:

- ① check error
- ② optimize
- ③ Convert to Assembly

* gcc -S File.c -o File.asm

③ Assembler

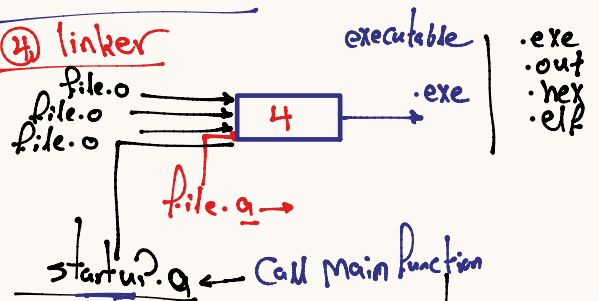


* Convert assembly into object

gcc -c File.c -o File.o

tool chain Part

④ linker



#include <stdio.h>

printf("ZMR");

#include <stdlib.h> → stdlib.a

Source File

Header File



Advanced Notes

① toolchains flags

* -E → Pre Processor only;

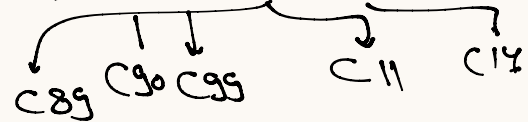
* -S → Pre Processor + Compiler

* -C → Assembler

* -Wall →

* -Werror →

* -std=c99



Single line Comment

// → c99 c11

→ c17

→ c89 → error

AVR → 32 kByte
→ 2 kByte

*Pre Processor directives

1) #include → * header file
 * text Replacement for file content

→ you can include header file from system
 ↳ use < >
 ex: #include <stdio.h>
 toolchain path

→ you can include user header file
 ↳ use " "
 Header file path

① Asboute

#include "D:\EmulC\session9\lab1\file.h"

disadvantage → not support to work with team

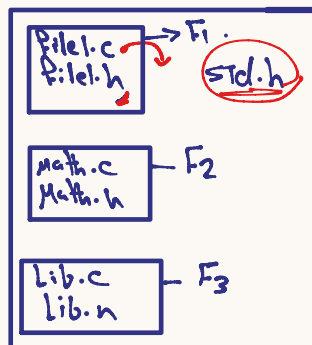
② Relative

File1.c

#include "File1.h"
 #include "../std.h"
 #include "../F2/Math.h"

Back one step

* → Back two step → ../..



lab2 → header file Function like macro

→ setBit (Reg, No)
 → clearBit (Reg, No)
 → ToggleBit (Reg, No)
 → GetBit (Reg, No)
BitMath.h

2) #define → object like macro
 ↳ function like macro

↳ text Replacement

* object like macro

#define ArrSize 3 ← No size

```
int main()
{
    uint8_t arr[ArrSize] = {0};
    for (i=0; i < ArrSize; i++)
    {
        for (j=0; j < ArrSize; j++)
        {
            // ...
        }
    }
}
```

typedef: Rename for Datatype
 typedef unsigned char us;

object like macro → create object

ex:.

① #define X 100
 int y = X;

Preprocessor → int y = 100;
 file1

Preprocessor	Compiler
int X = 50; int 100 = 50 ✓	int X = 50; give error

* Function like macro

#define Add(x,y) x+y
 int z = Add(5,7);

Preprocessor →
 file1 → int z = 5+7;

* Normal Function

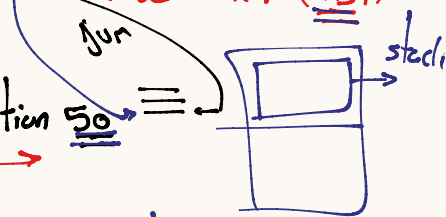
```
int Add(int x, int y)
```

```
{ return x + y; }
```

Assume function size in Rom (4B)

```
int Main()
```

```
{ // Call Add function
  Add(5, 3); }
```



Size Memory
total → 4B
Normal function
save memory

execution Processor

before Call →
Context switching

execute

Call

Back execute

Context switching

Normal function → take large execute time

* Function like Macro

```
#define Add(x, y) x + y
```

x + y

No size

```
int Main()
```

```
{ int R1 = Add(7, 3); }
```

```
int R2 = Add(5, 10);
```

```
int R3 = Add(10, 10); }
```

}

file .i

```
int R1 = 7 + 3;
```

```
R2 = 5 + 10;
```

```
R3 = 10 + 10;
```

4B size

4

4

4

12B

size

take large size

execution
fast

* [3] undef

warning → redefinition

ex: #define x 100

#undef x

→ Remove warning

#define x 200

#undef x

* Conditional Pre Processor

Pre Configuration

```
#define CarVersion 2
```

2

```
#if CarVersion == 1
```

```
MaxSpeed = 100;
```

```
#elif CarVersion == 2
```

```
MaxSpeed = 200;
```

```
#elif CarVersion == 3
```

```
MaxSpeed = 300;
```

```
#else
```

```
#endif
```

* elif → unlimited No
optional

* else → one time
optional

* endif → Mandatory
one time for if

if (version == 1) → True ✓

Car → version 1

{ maxSpeed = 100;

}

elseif (version == 2)

{ maxSpeed = 200;

}

→ Memory